Briefcase Software Development Plan

Fall 2026 Senior Project

Cara Babin, Amelie Dinh, Audrey Whitney-Miller, and Christian Ruiz Mendez

1. Plan Introduction

This Software Development Plan provides the details for the plan for the Briefcase transparency project which improves the existing workspace to give users more confidence in the generated information.

1.1. Project Deliverables

The Briefcase transparency project will integrate 3 subsystems into the existing Briefcase software to increase external visibility of text recognition confidence, quality of factual references in summaries, and effectiveness of the existing DocChat. Our 3 planned deliverables for this project are as follows:

➔ OCR – Expected completion: 10/19

Flagging areas from the source document with low text extraction confidence using visual indicators and a toggleable view of line accuracy probability.

➔ DocChat – Expected completion: 11/06

Implementing a more sophisticated chatbot to answer case-related queries with extensive citations for factual claims, which on-click will redirect the user to the location in the original transcript the information was found.

➔ Case Minimap Summary – Expected completion: 12/16

Reworking the case minimap feature generated for each transcript, including the examination summaries and headings; planned improvements include thorough citation for detail-oriented claims, visual indicators for sections with low confidence or which lack factual basis, and on-click redirection of citations to the source document for easier cross-referencing.

2. Project Resources

The Briefcase transparency project will build upon the existing Briefcase application and development structure. The project will be developed by Cara Babin, Amelie Dinh, Audrey Whitney-Miller, and Christian Ruiz Mendez. Individual responsibilities will be assigned according to the project schedule and the requirements of each subsystem.

2.1. Hardware Resources

The development of Brieface will primarily use the developer’s computers to host the project’s application and databases. Each computer will be used for writing and testing code, running the application locally, and accessing the project's existing infrastructure.

Briefcase can be run locally using Docker, which allows the frontend and backend services to execute on the same development machine. The project also uses existing database and hosting services to manage application data and processed transcripts.

2.2. Software Resources

The Briefcase AI Transparency project will use the existing Briefcase technology stack. The application consists of a React and TypeScript frontend, a Python and Flask backend, and several database and natural language processing components.

The following software resources will be used during development.

➔ React – Frontend framework
➔ TypeScript – Frontend development
➔ Python 3 – Backend development
➔ Flask – Backend web framework
➔ Tesseract OCR – Text extraction and confidence scoring
➔ Scikit-learn – Machine learning and clustering
➔ spaCy – Natural language processing
➔ PyTorch – Machine learning
➔ Hugging Face – Language models
➔ PostgreSQL – Relational database
➔ AWS RDS – Database hosting
➔ Elasticsearch – Text storage and search
➔ Firebase – Backend services
➔ Docker – Application containerization
➔ Docker Compose – Container management
➔ Node.js – JavaScript runtime

3. Project Organization

3.1. Project Management

This Briefcase project is organized into sprints developing three separate components which address separate features in the workspace. The tasks for each of these components will be organized using the project task manager inside of Github. Each component will be a parent task with the details of what needs to be done for the component in each sub-task. Each sub-task will be assigned to a team member who will carry out the task and change the status of the task once it has been completed. Within each parent task notes of the progress will be included along with a testing plan for each of the components.

3.2. Project Roles

To determine which tasks will be assigned to a team member we will use these roles: Backend Software Engineer, Frontend Software Engineer and ML Software Engineer.

Backend Software Engineer: Audrey

The role of the Backend Software Engineer will be to handle the database in which the documents are stored. In particular for this project, the engineer will determine what is

the proper way to store extracted case data inside of the database. This role will properly handle functionality to store and retrieve data from the ML Pipeline to the frontend.

Frontend Software Engineer: Christian

The role of the Frontend Software Engineer will be to handle the Briefcase workspace built in the React framework. This role will make the necessary UI changes needed for each feature and communicate with stakeholders what UI features are important to the customer. This role will be able to query the database for case data, present the data inside of the workspace, create new UI components, and test the new feature properly.

ML Software Engineer: Cara and Amelie

The role of the ML Software Engineer will be to make the necessary changes in the current ML pipeline to develop new features. The role will understand the necessary components that need to be modified. This role will use the existing pipeline to generate new information for features. This role will also optimize the current pipeline in order to make features more reliable and relevant for the users.

Project Schedule

This section includes the schedule for the Briefcase AI transparency project.

3.3. PERT / GANTT Chart

The GANTT chart for Briefcase was made with Figma. A link to the original figure is provided here.

3.4. Task / Resource Table

Note that some tasks are left TBD as teams and resources will be assigned when planning for that sprint begins.

Subsystem	Task	People	Hardware	Software
TE	Collect Confidence Scores from Text Extraction	All	N/A	Tesseract OCR, Python
TE	Store Confidence Scores in Database	Cara	N/A	ElasticSearch, Postgres, Python
TE	Display Confidence Score in Workspace	Amelie	N/A	React, Typescript
TE	Confidence Score Disappears on Outside Click	Christian	N/A	React, Typescript
TE	Highlight Low-Confidence Text	Audrey	N/A	React, Typescript
TE	Toggle Highlighting On/Off Control	Christian	N/A	React, Typescript

TE	TE QA & Integration Testing	All	N/A	N/A
DocChat	Collect Pages Used to Generate a Response	Amelie/Cara	N/A	TBD
DocChat	Implement Citations in Chat Responses	All	N/A	TBD
DocChat	Validate Citation Accuracy	Audrey	N/A	TBD
DocChat	Make Citations Clickable	Christian	N/A	TBD
DocChat	DocChat QA & Integration Testing	All	N/A	N/A
Minimap	Handle Unprocessable Sections with Fallback Indicator	Amelie	N/A	TBD
Minimap	Create New Workflow to Track Pages Used in Summary	Audrey	N/A	TBD
Minimap	Create Citations for Generated Summaries	Cara	N/A	TBD
Minimap	Create Clickable Citation Links to Transcript Pages	Christian	N/A	TBD
Minimap	Highlight Cited Text After Navigation	Christian	N/A	TBD
Minimap	Ensure Citation Navigation Does Not Reset Minimap	Christian	N/A	TBD
Minimap	Create Information Accuracy Confidence Score per Summary	Cara/Amelie	N/A	TBD
Minimap	Display Confidence Score Adjacent to Each Summary	Christian	N/A	TBD
Minimap	Recalculate Confidence Score on Summary Regeneration	Audrey	N/A	TBD
Minimap	Add Low-Confidence Warning Icon Below Threshold	Amelie	N/A	TBD
Minimap	Final Minimap QA & End-to-End Integration Testing	All	N/A	N/A